Manual

POSTMAIL

Version: 8.0.4
Date: August 21, 2026

Contents

1. Introduction

PostMail is a program for sending email to Microsoft Office 365. One of its key features is that it 's *independent* of external email programs like Microsoft Office, Outlook, or MS Mail. A major advantage is that the program is specifically designed to interact and communicate with other desktop programs .
In addition to Office 365, the legacy protocol SMTP (Simple Mail Transport Protocol) is also still supported.

Sending the email
This can be done in three different ways. Briefly summarized, these three ways are:

1st way
Launch PostMail.exe. A blank draft email message will be created and displayed in your default browser within Office 365. You can then finish and send the email there.
In the case of SMTP mail, PostMail itself shows you a dialog where you can create an email.

2nd way
An email is defined by defining it in a separate text file (with a *. txt extension). This file is passed as a parameter to PostMail.exe. PostMail reads from the definition file what it should do and then performs one of the following actions:

- A draft email is being created;
- The email will be displayed within Office 365 where you can complete and send the email;
- The email is displayed in a PostMail dialog, from which you can complete, finalize, and send the email;
- The email will be sent immediately, depending on the definition in the file.

3rd way
The email content can be passed to PostMail via the operating system's command line and stored in the email cache. The central concept of the email ID links the email data. For each ID, the data is stored in the user's roaming profile directory. After the email is sent, the stored data is deleted.
For further explanation, see the chapter “The email cache”.

2. Concepts

In the world of email, there are a number of concepts that are further defined and elaborated here to clarify the further manual.

The concept of 'address'
The remainder of this documentation refers to an "address." Addresses can take the following forms:

- Simple: Only the direct email username is provided to the provider, for example: jan.klaassen@example.nl
- Composite: An email user in brackets <>, preceded by an explanatory name. For example: Jan and Katrijn <jan.klaassen@example.nl>
- Multiple lists: Multiple email addresses, separated by ';' characters. For example: Jan and Katrijn <jan.klaassen@example.nl>; Brom Snor <brom.snor@ncrv.nl>; Malle Swiebertje <swieber@ncrv.nl>

The terms 'FROM', 'TO', 'CC' and 'BCC'
These terms further indicate the role of an "address." For example, in an email, there's always a sender, the "FROM" address.
The 'TO' addresses are the first recipients of the message;
The 'CC' addresses are the recipients who will receive an additional copy (Carbon Copy);
The 'BCC' addresses are recipients who also receive a copy, but this fact is not made known to the other recipients (Blind Carbon Copy).

The concept of 'subject' or 'topic'
Every email has a "subject". This is a short sentence summarizing the message (i.e., what it's about!). Note that PostMail allows you to send quite long subject lines, but some email clients (e.g., MS Outlook!) may not always display the complete sentence.

The concept of 'message'
The message is the actual content of the email. PostMail supports both plain text (RTF-formatted text) and HTML. Note that since 2018, MS Outlook no longer supports RTF-formatted text. Sending attachments with RTF remains possible .

The concept of ' delivery status '
Because email messages can be sent across multiple mail machines worldwide before reaching their destination, it's not always clear whether an email has actually reached its destination. Therefore, it's possible to request a return of a so-called delivered status message. You can request such a message for each individual message.

The concept of 'notification'
To ensure the program user that the email has been sent, a pop-up notification can be displayed in the foreground, indicating that the email has actually been sent. This notification occurs even if no email dialog is displayed and the email is sent automatically.

3. The structure of the mail file

The email file consists of two parts, separated by a line containing only the text "<BODY>" (or "<HTMLBODY>"). The section above this line contains the parameters specified using code words. Below the line containing the <BODY> character, the actual content of the email message is displayed. The message you can include here can be plain text, but it can also be formatted in HTML or RTF.

Possible parameters of the mail file are displayed in a table. This table is included four times, in the program's four supported languages (Dutch, English, French, and German). It also indicates whether the parameters are required or optional, and whether they may appear once or more than once (N times).

Table 1: Parameters Dutch

Parameter	Category	Time	Explanation
HOST:	Optional	1	The mail server used to send the mail
LOGIN:	Optional	1	Use <username>,<password> to log in to the mail server if the mail server offers this authentication functionality. Can contain the values "yes" or "no."
USER:	Optional	1	The name of the user on the mail server that will send the email. This is used if <login> is set to "yes."
FROM:	Optional	1	Sender's email address
TO:	Obliged	N	The recipient of the email. Must appear at least once.
CC:	Optional	N	Email address to which the CC message will be sent (=Carbon Copy)
BCC:	Optional	N	Email address to which the BCC message is sent (=Blind Carbon Copy. Recipients and CC'd recipients won't see this!)
SUBJECT:	Obliged	1	Description of the content of the email
PROFILE:	Optional	1	Name of the mail profile used to determine the sender and mail host
ERRORS:	Optional	1	If errors are detected (mail host down, email address incorrect), they can be displayed (the default) or suppressed by setting the value to "no." This is useful for batch processing large amounts of mail.
DIALOG:	Optional	1	If this parameter is set to 'yes', a dialog will be displayed before the email is sent.
READ-CONFIRMATION:	Optional	1	If this parameter is set to "yes," a read receipt for the email will be requested. However, there's no guarantee that it will be provided.
SENT - CONFIRMATION:	Optional	1	Can be set to “yes” to wait for an OK confirmation from the user after sending.
DELIVERED - STATUS	Optional	1	Delivery notification. “failure” Message that the email could not be delivered "success" Message that email has arrived on another mail server "delayed" Message that the email will be delivered later "header" Return only header “full” Entire email is reported back

			"never" Never send a message. If this parameter is not passed, it is assumed that one would always like to receive a 'failed' notification. Default: Return the header on failure
PROGRESS:	Optional	1	If "no" is selected, no progress dialog will be displayed while the email is being sent. If "yes" (the default value), progress will be displayed.
DELETE:	Optional	1	Select "Yes" to delete the mail definition file after successfully sending the email. By default, the definition file remains intact.
EDITSUBJECT:	Optional	1	Can be set to “yes” so that the sender can edit the subject line in the dialog
EDITBODY:	Optional	1	Can be set to “yes” so that the sender can edit the message text in the dialog
NOTIFY:	Optional	1	If set to "yes," the recipient will then be asked to allow a receipt notification to be sent back to the recipient.
WRITE-CHANGES:	Optional	1	Can be set to “yes” to save subject or message changes to a separate file.
NOPWDCRYPT:	Optional	1	No value. If this parameter is present, an unencrypted password can be specified with the "PASSWORD" parameter. *Use only for testing purposes!*
PASSWORD:	Optional	1	Password of the user on the mail server that will send the email. Used if <login> is set to "yes."
ATTACH:	Optional	N	Pathname to a file to be attached
IMPORTANCE:	Optional	1	Priority of the email Can contain the values “high”, “low”, or “normal”.
LANGUAGE:	Optional	1	Interface language. Allowed values are 'Nederlands', 'English', 'Francais', and 'Deutsch'.
TIMEOUT:	Optional	1	Connection timeout to the mail server. Permitted values are between 2 and 20 (inclusive).
LOGLEVEL:	Optional	1	Logging level in file "%TMP%\Logfile PostMail.txt" 0 -> No logging 1 -> Server responses are logged 2 -> Client requests are logged 3 -> All traffic is logged in HEX format

An example

Here is an example of the contents of an email file:

```
HOST:smtp.organization.eu
FROM:Edwig Huisman<edwig.huisman@organisatie.eu>
TO:GE User <ge.bruiker@organisation.nl>
ATTACH:C:\TFS\CM\PostMail\Releasenotes postmail.txt
ATTACH:C:\TFS\CM\PostMail\mail.txt
SUBJECT:The postmail program has been updated again!
DIALOGUE=yes
PRIORITY: high
READ CONFIRMATION: yes
DELIVERY STATUS: Failed
NOTIFY: yes
<BODY>
Dear all,

This is a sample email that was sent with the updated
version of PostMail. Several improvements have been made
compared to the previous version:
- Integration with Microsoft Office 365

How beautiful is that?

Kind regards,
Edwig Huisman
```

4. OFFICE 365 configuration

To configure the integration with Office 365, a " PostMail.json " file has been added to the product. Here is an example of such a file.

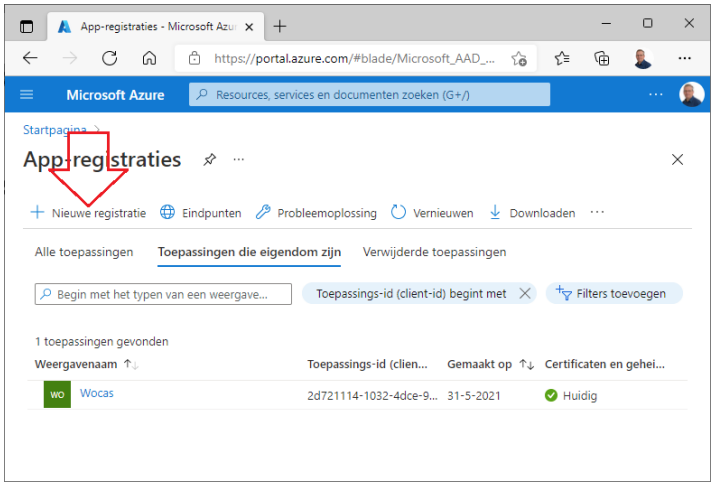
```
{
       "type": "Office365",
       "token-server" : "https://login.microsoftonline.com/%s/oauth2/v2.0/token",
       "azure-tenant" : " qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq ",
       "app-identity" : " yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy ",
       "app-secret" : " xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx ",
       "app-scope" : "https://graph.microsoft.com/.default",
app browser: browse
       robot: {
                    "uses-account": "noreply@organisation.nl",
                    " uses -alias" : "Task processor"
       }
}
```

The different parameters in this file are explained below:

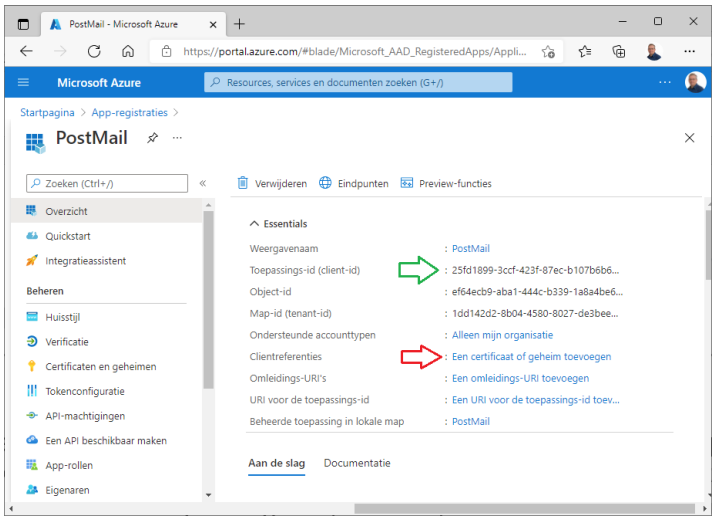
Parameter	Operation
type	Indicative: used in notifications and logs
token server	Microsoft's OAuth2 authentication server for Office 365
azure -tenant	Your organization identification number at Microsoft Azure
app identity	PostMail registration number for your organization in Azure
app- secret	Your PostMail password for your organization in Azure
app scope	Indication that PostMail is logging in for MS-Graph
app browser	Can contain the values 'browse' or ' msg '. Determines whether the draft opens in the browser, or whether only a message follows.
uses -account	Disposition account that sends the mail (instead of the real one)
uses -alias	Displayed user in mail programs for the robot user

To populate this configuration file for your organization, you need to perform a number of actions in the Microsoft Azure portal. These actions must either be performed by the Azure administrator (most efficient) or approved afterward by the Azure administrator via a web link.

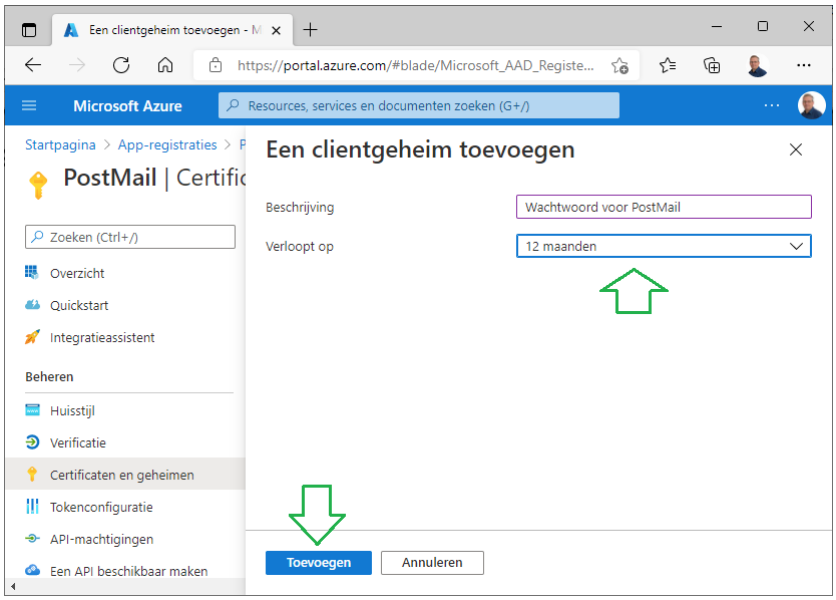
Follow the steps described here:
1) Go to the Azure portal (https://portal.azure.com)
2) Go to “App Registrations”
3) Select “+ New registration” at the top left



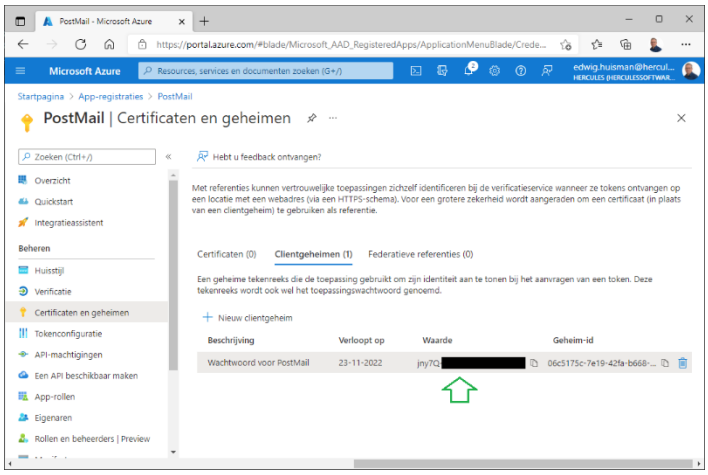
4) In the next window, in the first field under "Name," enter only "PostMail" and ignore all other options in this window. Click the "Register" button at the bottom.
5) You will now arrive at a page for your "PostMail" application. Directly below the name is the "Application ID (Client ID)" field. This is followed by a GUID value. Note this value. Two lines below, you will find the "Folder ID (Tenant ID)." Note this value as well. You will need it for the JSON configuration file.
6) Under “Client Credentials” select the option “Add a certificate or secret”;



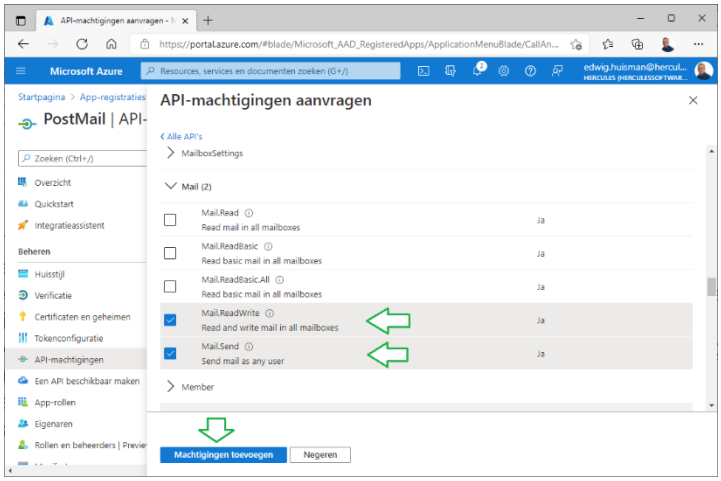
7) On the next page, select "New Client Secret" from the "Client Secrets" tab . Enter a description, for example, "Password for PostMail" and a password period. Options include 6, 12, 18, or 24 months. Then click "Add."



8) NOTE: You will now be shown the password once. Note the password under the Value column, or use the Copy button after the value to copy the password to the Windows clipboard.

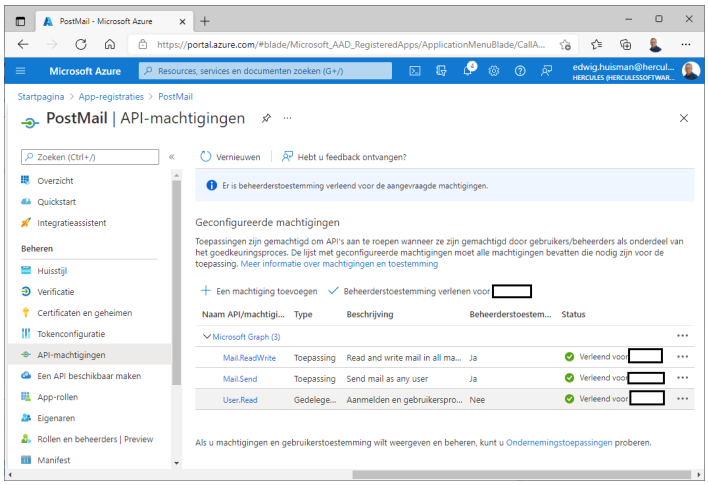


9) You now have enough data for the configuration file;
10) Now select “API Authorizations” in the menu on the left
11) Use the “+ Add a Permission” button, and use the big wide button at the top “Microsoft Graph”;
12) In the next window, select the button on the right labeled “Application Permissions”;
13) Scroll down to the “Mail” option and expand it;
14) Select the last two options “ Mail.ReadWrite ” and “ Mail.Send ” and press the “Add Permissions” button;

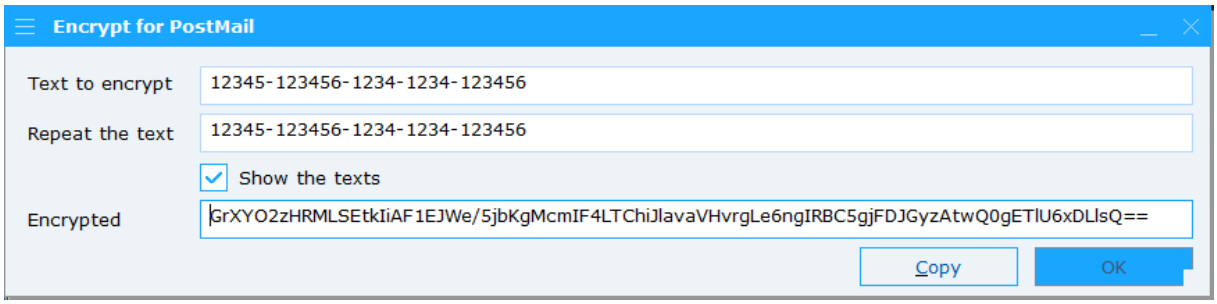


15) You will now see a window with an overview of the permissions. However, the permissions are not yet activated. If you are the Azure administrator, you can do this for the two email permissions using the "Grant administrator consent" button. If not, the button will open a pop-up with a web link that you can email to your Azure administrator. The administrator can follow these links from their mailbox and will be taken to an acceptance dialog where the permission can be granted.
Note that PostMail can only write email to the mailbox of the currently logged-in MS-Windows AD user. PostMail cannot read or write email to anyone else's mailbox!

16) The rights should then look like this:



17) You can now enter the collected values into the JSON file. To do this, use the "PostMailEncrypt" application, which you can find in the working directory.
Enter the following values, using the encrypted values for the JSON file. Do this for the GUID values of:
- The Azure tenant ID
- The application client ID
- The password secret
Below you can see an example of the PostMailEncrypt application



18) Fill in the three values in the JSON file for “ azure -tenant”, “app- identity ”, and “app-secret ”, and save the file.

5. Command line syntax

The PostMail.exe program can be started with a text file containing the definition of an email as follows:

```
POSTMAIL.EXE [options] {[definition.txt]|[mail-part]}
```

This means that you can call the program with zero, one or more options followed by either a definition file (see Chapter 3) or a mail cache command.

All options are described further below:

Option	Explanation
/H, /?	Help page with all options and command line syntax
/N	Interface in Dutch
/E	Interface in English
/F	Interface in French
/D	Interface in German
/S	The program was started by a server. Under no circumstances should interfaces and/or message dialogs be displayed. If you start this program and the program still displays an error message, the program will still open a log file to write the messages there.
/T	Interface starts up in Text mode, you can choose plain text or RTF text, but no HTML
/L	A log file will be opened at level 2 (log everything). This option can be useful for debugging your program to ensure all mail is being passed through correctly. The log file will be opened immediately upon startup in your temporary directory and named "LogfilePostmail.txt." (See also below under "Debugging").
/A:"logfile "	The log is not written to a separate file, but appended (append) to an existing log. The full syntax of this option is as follows: / A:"Absolute path to file". This allows you to append the generated log to another application's log.
/X	The log is opened at the highest level (He X adecimal tracing). All communication with the mail server is logged, including the secure TLS connection.
/P	Before sending the mail, the profile selection dialog is first displayed, allowing the user to select a profile (mail host and 'sent on behalf of') *)
/P:<n>	The profile number <n> is used to send the email. (1 = first profile). The mail host and sender information from that profile will then be used to send the email. *)
/P:profile	For software that stores its own profile names, it is possible to specify the profile name using "/ P:profilename ". *)
/C	Start the program in configuration mode and open the profile management dialog directly. After managing the profiles, the program will close immediately.
/O	Immediately start using the PostMail Outbox so you can view and/or manage its contents . Closing this dialog will also immediately terminate the program.
/V	The program will only be used as a viewer of an existing definition file. The "Send" button will be invisible, so the email cannot be sent, and the DIALOGUE, CHANGE SUBJECT, and CHANGE MESSAGE options in the definition file will not be activated.
/K	The program will be displayed purely as an encryption dialog. This is

	similar to the "PostMailEncrypt" program.
/Q:ping	Command to ping the current user to the ERP application. Tests whether the user is still logged in.
/U:login	Command to log a user (User) into the ERP application. Can be used if the ping command (see the /Q option) returns no positive result
/R:ready	Command to report the mail as ready. The ERP program can then increment the 'sent mail counter'.
/Z:profile	Command to report the change of the last used mail profile to the ERP program
Mailpart	**Explanation**
/ID<n><OPTION>	Add an option to the email in the mail cache with id <n>. By issuing multiple commands, an email will be built in the mail cache until you issue one of the following commands. **)
/ID<n>	Send (only) the email with the mail ID <n> **)
/ID	Send all emails in the mail cache (empty the mail cache) **)

*) If one of the '/P' options is used and the mail is sent correctly, the program will return the profile number via the command line return value (see below).

**) For the operation of the /ID option, please refer to Chapter 7 on the ID cache of the PostMail program.

Command line return values
To facilitate communication with the calling program, PostMail returns a status code to the calling program using the return value. The possible status codes are shown in the table below:

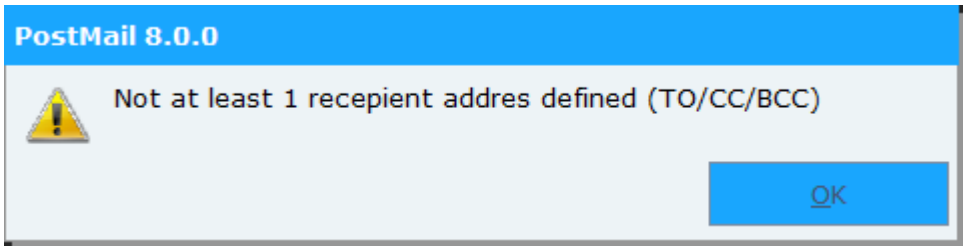
Table 2: command line status codes

Status code	Explanation
0	Mail has been sent correctly
1	The user clicked the "Cancel" button. No action was taken.
2	The program was started as a viewer. No actions were taken.
3	Tried to send mail, but errors occurred.
4	Help message displayed and stopped.
5	No mail file with parameters was specified. There is nothing to send.
6 to 10	<(not yet) in use>
11	Mail sent correctly and Mail Profile "1" was selected (return value – 10)
Higher than 11	Mail sent successfully and Mail Profile <n> was selected (return value – 10)

6. Detecting errors

Errors are initially reported in the foreground, as the program wasn't started in the background by a server. All errors are reported to the user collectively through a dialog box. For example:

Figure 10: Errors while sending



Because errors can be suppressed, and because the program can be started by another - program in the background, it is possible to use a logbook.
However, the option to turn on the log is only intended for temporary use, as the log will always appear on the desktop immediately after it is finished, thus discouraging turning on the log for daily use.

For background programs, it's possible to suppress the log display by using the "/A:" command line option. This option appends the log output to an existing log.

The log can be enabled using the command-line parameter (/L) or the "LOGLEVEL" parameter in the mail definition file. There are four logging levels in total:

0 Only the reading of the definition file and whether or not it has been sent are logged.
1 Outgoing commands to the SMTP server are also logged.
2 The responses from the SMTP server are also logged
3 Everything is logged, including error information. If an error occurs, the system automatically switches to this logging level internally.

The log file shows a number of sections in succession:

- The imported mail definition file, marked with “<<” end-of-line markers;
- The read state of the definition, so that it can be determined whether the definition file has been interpreted correctly;
- A transcript of the connection to the SMTP server and the protocol exchange between the client (PostMail) and that server.

An example of a log file:

```
PostMail 6.3

Start of the log file
------------------------------------------------------------
C:\Batches\mail.txt
------------------------------------------------------------
HOST:mailhost.hetnet.nl<<
FROM:Edwig Huisman <edwig.huisman@organisation.nl><<
TO:Bart in Molder <bart.te.molder@organisation.nl><<
ATTACH:d :\ projects \PostMail\Releasenotes postmail.txt<<
ATTACH:d :\ projects \PostMail \mail.txt<<
ATTACH:d :\projects\PostMail\Messages.cpp<<
SUBJECT:The Postmail program has been updated again!<<
PRIORITY: high <<
NOTIFY:yes <<
DIALOGUE:yes <<
READ CONFIRMATION: yes <<
<BODY><<
Dear Bart,<<
This is a sample email sent with the updated<<
version of PostMail. Several improvements have been made<<
compared to version 2.x<<
- No more WinBatch , but real C++ with many error messages (see release notes )<<
- Ability to debug and troubleshoot the mail protocol!!<<
- More and custom parameters in the mail file.<<
- Multilingual. (not so shocking).<<
- Extended command-line syntax.<<
Because the program is no longer in WinBatch , it has become much easier<<
to maintain and/or expand the program.<<
<<
Could you take advantage of this?<<
<<
Kind regards,<<
Edwig Huisman<<
------------------------------------------------------------
INTERNAL STATE OF THE MESSAGE AS READ:

Mailhost : mailhost.hetnet.nl
FROM: Edwig Huisman <edwig.huisman@organisation.eu>
TO: Bart te Molder <bart.te.molder@organisation.nl>
Subject: The postmail program has been renewed again!
Show dialog: yes
Show progress: yes
Show errors at end: yes
Notify sender in BCC list: yes
Importance: high
Delivery status notification: Full, Failure
Mail disposition notification: yes
ATTACHMENTS:
File: d:\projects\PostMail\Releasenotes postmail.txt
File: d:\projects\PostMail\mail.txt
File: d:\projects\PostMail\Messages.cpp

THE MESSAGE BODY:

Dear Bart,
This is a sample email that was sent with the updated
version of PostMail. Several improvements have been made
compared to version 2.x
- No more WinBatch , but real C++ with many error messages (see release notes )
- Ability to debug and troubleshoot the mail protocol!!
- More and custom parameters in the mail file.
- Multilingual. (not so shocking).
- More extensive command-line syntax.
Because the program is no longer in WinBatch , it has become much easier
to maintain and/or expand the program.

Could you take advantage of that?

Kind regards,
Edwig Huisman
------------------------------------------------------------
SERVER: 220 hnexfe08.hetnet.nl mailhost.hetnet.nl Thu , 13 Jul 2006 08:02:31 +0200
CLIENT: EHLO PC-EHUISMAN
SERVER: 250-hnexfe08.hetnet.nl Hello [193.173.228.130]
SERVER: 250-TURN
```

```
SERVER: 250-ATRN
SERVER: 250-SIZE 7864320
SERVER: 250-ETRN
SERVER: 250-PIPELINING
SERVER: 250-DSN
SERVER: 250-ENHANCEDSTATUCODES
SERVER: 250-8bitmime
SERVER: 250-BINARYMIME
SERVER: 250-CHUNKING
SERVER: 250-VRFY
SERVER: 250-X-EXPS LOGIN
SERVER: 250-X-EXPS=LOGIN
SERVER: 250-AUTH LOGIN
SERVER: 250-AUTH=LOGIN
SERVER: 250-X-LINK2STATE
SERVER: 250-XEXCH50
SERVER: 250 OK
CLIENT: MAIL FROM:<edwig.huisman@organisatie.eu> RET=HDRS
SERVER: 250 2.1.0 edwig.huisman@organisatie.eu....Sender OK
CLIENT: RCPT TO:<bart.te.molder@organisation.eu> NOTIFY=FAILURE
ORCPT=rfc822;bart.te.molder@cent
ric.nl
SERVER: 550 5.7.1 Unable to relay for bart.te.molder@organisation.nl
The recipient's address was not accepted:
Could not send the SMTP email message
550 5.7.1 Unable to relay for bart.te.molder@organisation.nl
CLIENT: QUIT
SERVER: 221 2.0.0 hnexfe08.hetnet.nl Service closing transmission channel
------------------------------------------------------------
End of the logbook
```

7. The email cache

You can specify an email ID for the email cache on the command line. This ID allows you to build the email in phases. One of the parameters from Table 1 on page 2 can be specified for each command-line call.
After all parameters have been passed for the email, it is possible to send the email by specifying the email ID, without a parameter.
A final option is to pass just the “/ID” option, which will result in the entire email cache being sent to the mail server.

Example:

```
Postmail.exe /S /ID:300 /host:smtp.organisatie.eu
Postmail.exe /S /ID:300 /from:edwig.huisman@organisatie.eu
Postmail.exe /S /ID:300 /to:bart.de.programmeur@organisatie.eu
Postmail.exe /S /ID:300 "/subject:New version of Postmail"
Postmail.exe /S /ID:300 "/dialog:yes"
Postmail.exe /S /ID:300 "/body:Postmail once again renewed"
Postmail.exe /S /ID:300 "/body:Read the release notes"
Postmail.exe /S /ID:300 "/body:for the new features"
Postmail.exe /S /ID:300
```

It is the last line that sends the email with ID 300 to the mail server.
During the email build process, the email is stored in the following location:

```
C:\Users\<Username>\AppData\Roaming\Hercules\Postmail\email_<ID>.ceml
```

This makes it possible to create and save multiple emails and, if necessary, send them in bulk. The command:

```
Postmail.exe /ID
```

Ensures that the total email cache built up under “Roaming\Hercules\ Postmail ” flushed to the mail server. All emails will be sent and placed in the outbox .

8. Default mail servers and profiles

It's mandatory to create a small text file named "Postmail.Mailservers.ini" in the same folder where you install the "PostMail.exe" program. This file contains the default mail servers known to your system. The first mail server listed in this file is the default mail server.

```
Default mail servers in this list
; First mail server is the default
;
smtp.organisatie.nl
```

Comment lines can be added to the file, which PostMail will ignore when reading. These comment lines must begin with the ';' or '#' character. The remaining lines specify one mail server per line. The first is the default mail server.

This file is used when managing profiles in the Profiles dialog. The mail servers listed here will be displayed in a mail server combo box when managing profiles.

The mail server is also checked against this file when sending the email. If the mail server in the mail definition isn't on the list of these servers, the email will be sent to the first (default) mail server.

If the file doesn't already exist when the program is launched for the first time, it will be created. Of course, no mail servers will be listed yet. You'll need to create them yourself.

Managing the profiles
Chapter 4 describes how to create and manage profiles.
If there is no default profiles file present when the PostMail program is first started, a default 'Postmail.DefaultProfiles.xml' file will be created in this directory – provided permissions are granted – containing one default ' noreply ' profile and one profile with the user's data.

Other user profiles are created in the user's so-called roaming profile folders under:
`C:\Users\<user>\AppData\Roaming\Organisation\Postmail\Profiles.xml`

Or for Citrix administrators in the following format
`%APPDATA%\Organisation\Postmail\Profiles.xml`

The default user profile contains the macros "@FROM" and "@USER." The actual values of these macros are populated via Active Directory (AD), making it virtually unnecessary for users to create additional profiles beyond these two default profiles.

9. Mailboxes

You can configure (default) mailboxes within PostMail. There are several ways to store and retrieve your mail. You can set up a general mailbox for the entire organization, just as you would on a mail server, or you can choose to let users manage this themselves.

If you do NOT configure anything, the defaults are as follows:
a) The shared mailbox is turned off
b) The central log file for emailing is turned off

In the default configuration file "PostMail.DefaultProfiles.xml" the following variables are included FOR the first profiles:
- SharedOutbox Where the shared Outbox is located
- CentralLogfile Where the central logbook is located
- DefaultFont Which font is used in the mail dialog

If the default configuration file "PostMail.DefaultProfiles.xml" in the current software folder is editable, these variables can be modified by the current user. This usually requires running the program "PostMail.exe" with "As Administrator" privileges.

Example:

```
< PostMailProfiles >
< SharedOutbox >M:\org\Data\ MailBox \</ SharedOutbox >
   <CentralLogfile>M:\org\Data\MailBox\Postmail_logfile.txt</CentralLogfile>
< DefaultFont >Open Sans;10;#000000</ DefaultFont >
  <Profile>
...
</Profile>
</ PostMailProfiles >
```

All user mailboxes are located in the SharedOutbox in a separate subdirectory created from the AD name and user name. For example:
"M:\org\data\MailBox\ORG_user.name\”

You should keep in mind that the folders you specify here are writable by all users, and that the users have or will be given write access to the central log.

Own choice:
A user can choose their own mailbox, but only if the organization's shared mailbox is NOT enabled. If it IS, you can only set the default number of days to search.

If the organization does NOT have a shared mailbox, you can specify in your personal profile whether you want something different. The options are:
a) NOTHING
b) Default in the roaming profile.
c) your own personal folder

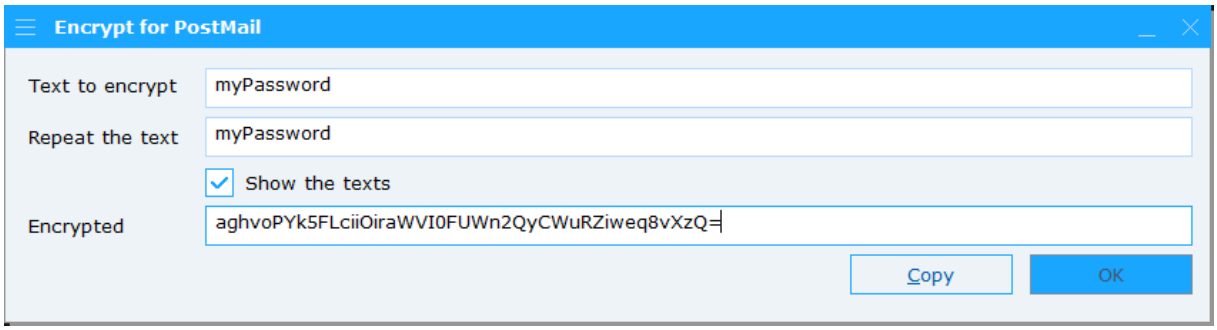
Personal settings are saved in the roaming profile. These settings—and possibly the mailbox as well—are stored in the user's roaming profile in the "Hercules\ Postmail " folder.
The roaming profile is 'loaded' in a Citrix environment so that users ' take' their mail settings with them to another machine.

NOTE 1: Option b) places a heavy load on the login system, as you're always dragging your personal mailbox along with you when you log in. In such a case, c) is a much better option.

NOTE 2: System administrators and functional managers often ask if it's possible to configure these settings for all users at once. However, this isn't possible within PostMail, as the mailbox is the user's personal domain. Legally, the employer has no control over this. The best way for (system) administrators to influence this at once is to set up a central mailbox folder.

10. Authentication by the email server

An SMTP mail server can perform user authentication. If a mail server uses this mechanism, it guarantees a higher level of privacy and security.
PostMail can deliver mail to SMTP servers that require user authentication. To do this, the definition file must specify a mail ID and password, along with the requirement to use a login. The password entered in the email file must be encrypted using the included application, "PostMailEncrypt". Enter the password in this application, and the encrypted version of the password will be displayed in the "Encrypted" field. You can select this encrypted text from the text field or use the "Copy" button.



This will provide you with enough information to encrypt your login using the commands 'LOGIN' (use the login), 'MAILID' (the mail server account), and 'PASSWORD' (the password you just encrypted).
You include the data in the mail definition file as follows:

```
LOGIN:yes
MAILID:org\jklaassen
PASSWORD:uyWNJ46zltFZRSUJj6X/5pqAxu+oMTtzlkj4KrD2lWg=
```

Within the PostMail program, the password is decrypted and immediately re-encrypted in BASE64 format before being passed to the SMTP server. This means no unencrypted passwords are ever sent over the wire.
However, the content of the email message always travels unencrypted over the network to the SMTP server, regardless of the authentication mechanism. [1])

However, the login mechanism is only used if the SMTP server indicates that this login mechanism is actually offered as an option on that SMTP server.
Additionally, an SMTP server can be configured in such a way that an email connection is not accepted if no one logs in.

[1]Microsoft's Exchange server, in addition to the LOGIN authentication mechanism, also offers NTLM or GSSAPI login options within its SMTP functionality. For both other mechanisms, the login itself is encrypted, but the message content is always sent unencrypted.

All things considered, the following options exist:

	Mail server does not offer LOGIN	Mail server offers LOGIN as an option	Mail server always requires at least LOGIN
email without login	Mail is being sent	Mail is being sent	Mail server refuses to send the email
email with LOGIN	Mail is being sent	Logging in. If successful, the email will be sent.	Logging in. If successful, the email will be sent.

11. DKIM , SPF and DMARC security

A modern way to protect against spam email and other unwanted attention, such as spoofing or phishing , is to cryptographically secure email. This is done by adding cryptographic features to the email as it is sent. The public key for SPF security is passed via the DNS (Domain Name Service) in the DKIM record.
The DMARC record in the DNS indicates what should happen to non-cryptographically secured mail.

PostMail offers two delivery modes. The following applies:

- Sending via MS-Graph and Office 365 (configured with "postmail.json"), as described in Chapter 4 of this manual, will properly secure the mail with SPF and DKIM. As such, the mail will arrive as "secure and verified";
- Sending via the (S)SMTP protocol. In this case, PostMail handles the sending itself, and the email is **NOT** cryptographically secured.

If you use SMTP and your organization has secured email with DKIM/DMARC, it's important to configure the DMARC policy correctly. If you don't, emails sent with PostMail won't arrive.

DMARC has a so-called 'policy' (p=…) and ' subpolicy ' (sp =…) field, which can be set to three values, namely:

- 'none' Mail always gets through, you have to check it manually.
- ' quarantine ' Not cryptographically signed mail goes into the quarantine box
- ' reject ' Not cryptographically signed mail is discarded

It should be clear that the last of these three will ensure that the emails will never be visible, not even to the mailbox administrator.

If the organization chooses to send incoming, uncryptographically signed mail to the quarantine box, this means that quarantine rules must be defined, because the mail will remain there and eventually (usually after 30 or 45 days) be discarded.

Practical experience shows that for SMTP PostMail to function correctly, the DMARC record must be set to either "none" or " quarantine ." In the latter case (quarantine), it is recommended to thoroughly investigate and adjust the quarantine rules!

APPENDIX: Changes compared to previous versions

Changes prior to version 8.0 have been removed from this document.

Releasenotes 8.0.0
- The interface has been adjusted to support high DPI (> 96 DPI) monitors;
- The cryptographic encryption of profile management passwords and passwords in email definition files has been updated. The encryption has been upgraded to AES 256-bit encryption;
- All encryption is now done through the 'PostMailEncrypt' application;
- Complete documentation of all mail definition files';
- All commands are now definable in four languages (Dutch, English, French and German);
- Where previously only the main window was in four languages, now all other dialogs and windows are also in four languages.

Releasenotes 8.0.1
- A bug in the determination of the current UPN (email identification on Azure's Entra-ID) has been fixed for sending email via MS-Graph.

Release Notes 8.0.2
- Fixed a bug regarding reporting an error message for an incorrect email address when emailing Office 365. The application crashed with an email address containing a non-existent domain name after the '@';
- Fixed a bug regarding reporting a non-loadable attachment when emailing Office 365. An error message is now generated per attachment, plus an error message for all attachments together;
- If the application is started as a server application with "/S", any errors are now always written to BOTH the Windows event log AND a log file in the %TMP% directory;
- Additional detection has been built in for starting from an NT Service or a headless IIS service. In this case, the "/S" server option is automatically enabled so that foreground notifications cannot cause the PostMail program to hang;
- - There are applications and functions that insert extra <CR> or <CR><CR><LF> characters into the mail definition files. From now on, this will be checked by scanning and removing both types of characters independently of each other at the end of every line.

Release Notes 8.0.3
- If email was sent via SMTP and attachments larger than 16K bytes were sent, these attachments were corrupted after this point. As a result, both text and binary files did not arrive correctly at the recipient.

Release Notes 8.0.4
- Small corrections for bugs while working with SMTP and profiles where fixed